# Gas Sensor Array Drift

Final Submission Materials & Methodology Report

## 1. Final Notebook / Methodology Report

### Complete Description of the Approach

The solution addresses the gas sensor drift and distribution shift challenge through a robust two-stage classification pipeline.

- **Stage 1 (Base Pipeline):** A multi-class ensemble that applies univariate moment-matching drift alignment, evaluates models using a forward-chaining temporal split, and refines predictions via iterative pseudo-labeling.
- **Stage 2 (Rescue Cascade):** A binary classification cascade targeting classes 3 and 5. Because class 3 tends to act as an uncertainty sink that misclassifies true class 5 samples, Stage 2 selectively overwrites class 3 predictions to class 5 when the binary model is highly confident and the sample concentration is physically plausible based on training bounds.

### Data Cleaning and Preprocessing

Raw sensor features are winsorized to cap extreme outlier values. This winsorization leverages robust z-scores calculated via the median and median absolute deviation (MAD). To counter sensor drift over time, a univariate moment-matching alignment is applied, shifting the mean and variance of the training features to align with the test set distributions. Concentration and log-concentration features are explicitly excluded from this alignment as they relate to experimental dosing protocols rather than sensor degradation.

### Feature Engineering

The engineered feature space builds upon the 128 raw sensor responses:

- **Concentration Scaling:** Raw features are scaled by concentration to yield `cnorm` features.
- **Statistical Aggregations:** Per-position statistics (mean, standard deviation, range) and per-sensor-type statistics are derived.
- **Ratio Features:** Cross-type ratios and adjacent-sensor ratios are calculated for the most informative sensor positions to capture relative response patterns.
- **Quality Signals:** An outlier count feature is engineered to explicitly signal noisy or anomalous measurements.
- **Stage 2 Isolation:** The rescue cascade relies strictly on features derived from the TGS2602 (sensor2) units, which maintain the strongest resolving power between classes 3 and 5.

## Validation Strategy

Given the temporal nature of the drift, standard random cross-validation is inappropriate. Instead, the validation schema employs:

- **Forward-Chaining:** Temporal splits (train $\leq 6$ to validate on 7, train $\leq 7$ to validate on 8, train $\leq 8$ to validate on 9) are averaged.
- **Batch 7 Holdout:** A dedicated holdout validation on batch 7 is combined with the forward-chaining score for final model selection.
- **Leave-One-Batch-Out (LOBO) OOF:** Out-of-fold probabilities used for calibration and pseudo-labeling are generated via a LOBO GroupKFold scheme across the 9 training batches.

## Models Tested and Final Model Selection

XGBoost, CatBoost, and LightGBM were comprehensively evaluated in Stage 1. The top two models from the time-split validation serve as the primary base models. Additionally, four independent LightGBM "expert" models are trained—each strictly limited to a single sensor type to capture sensor-specific signal degradation.

## Ensembling and Post-Processing

- **Stacking:** A multinomial logistic regression meta-learner stacks the LOBO OOF probabilities of the two main models and the four expert models.
- **Calibration:** Class weights are optimized using a Nelder-Mead algorithm with L2 regularization to maximize the macro-F1 score without oscillating.
- **Pseudo-Labeling:** Three rounds of pseudo-labeling adapt the models to the test set batch.
- **Concentration Prior:** A physical concentration ceiling penalty is integrated. Predictions extrapolating beyond the maximum concentration observed for a given class in the training data receive a soft cost-penalty.
- **Stage 2 Ensemble:** The binary rescue cascade uses a 50/50 soft-voting ensemble of XGBoost and CatBoost.

## Key Results, Observations, and Limitations

The TGS2602 sensor degrades the fastest but is uniquely capable of distinguishing between classes 3 and 5. Because early-batch representations of class 5 contaminate later drift states, the Stage 2 cascade is trained exclusively on recent batches (7, 8, and 9). A minor limitation is a slight circularity in validation where batch 9 is used for both forward-chaining selection and OOF calibration. Furthermore, sensor measurements at low concentrations ($< 50$ ppm) were identified as highly sensitive to relative drift correction errors.

## 2. Complete Source Code

The complete logic for data loading, preprocessing, feature generation, model training, validation, inference, and post-processing is contained within two Python scripts:

- `gas_sensor_pipeline.py` (Stage 1 Base Pipeline)
- `stage2_cascade_3_vs_5_2.py` (Stage 2 Rescue Cascade)

Both scripts are submitted alongside this document.

## 3. Final Submission / Prediction File

The pipeline executes sequentially to produce the prediction files:

- Intermediate output: `submission_final.csv`
- Final submitted file: `submission_final_cascaded.csv`

## 4. Processed / Cleaned Dataset(s)

No static intermediate datasets are provided. All processed features, drift-aligned matrices, and pseudo-labeled additions are dynamically generated in-memory during the execution of the provided Python scripts.

## 5. README / Reproduction Instructions

**Required Files:**

- `gas_sensor_pipeline.py`
- `stage2_cascade_3_vs_5_2.py`
- `train.csv` (Competition training dataset)
- `test.csv` (Competition test dataset)

**Execution Order:**

1. Run `python gas_sensor_pipeline.py`. This executes the base pipeline and outputs `submission_final.csv`.
2. Run `python stage2_cascade_3_vs_5_2.py`. This script reads the intermediate output, applies the binary cascade, and generates the final prediction file `submission_final_cascaded.csv`.

**Dependencies:** Python 3, NumPy, Pandas, XGBoost, CatBoost, LightGBM, scikit-learn, SciPy.

**Important Settings:** All models and validation splits utilize a fixed random seed (`RANDOM_STATE = 42`) to ensure reproducibility. For a faster execution check (non-production), the environment variable `GSD_FAST_DEBUG=1` can be set.

## 6. Final Model Information

- **Stage 1 Base Ensemble:** Dynamically selects between XGBoost, CatBoost, and LightGBM based on temporal validation. It incorporates four LightGBM sensor experts and is unified by a Logistic Regression meta-learner. Base models employ learning rates around 0.03 to 0.035 and maximum depths of 6 to 7.
- **Stage 2 Cascade Ensemble:** A 50/50 soft-voting ensemble comprising an XGBoost Classifier (300 estimators, depth 5, learning rate 0.05) and a CatBoost Classifier (300 iterations, depth 5, learning rate 0.05).

## 7. Required Disclosure

- **External Datasets:** No external datasets were searched for, matched, or utilized. All concentration ceilings and data bounds are derived exclusively from the provided `train.csv`.

- **Pretrained Models:** No external pretrained models or AutoML systems were used. All models are trained locally from scratch.
- **Test Labels:** No prohibited label recovery techniques or external source dataset information were used to determine class predictions. The final classification avoids enforcing an aggregate class distribution on the test set.